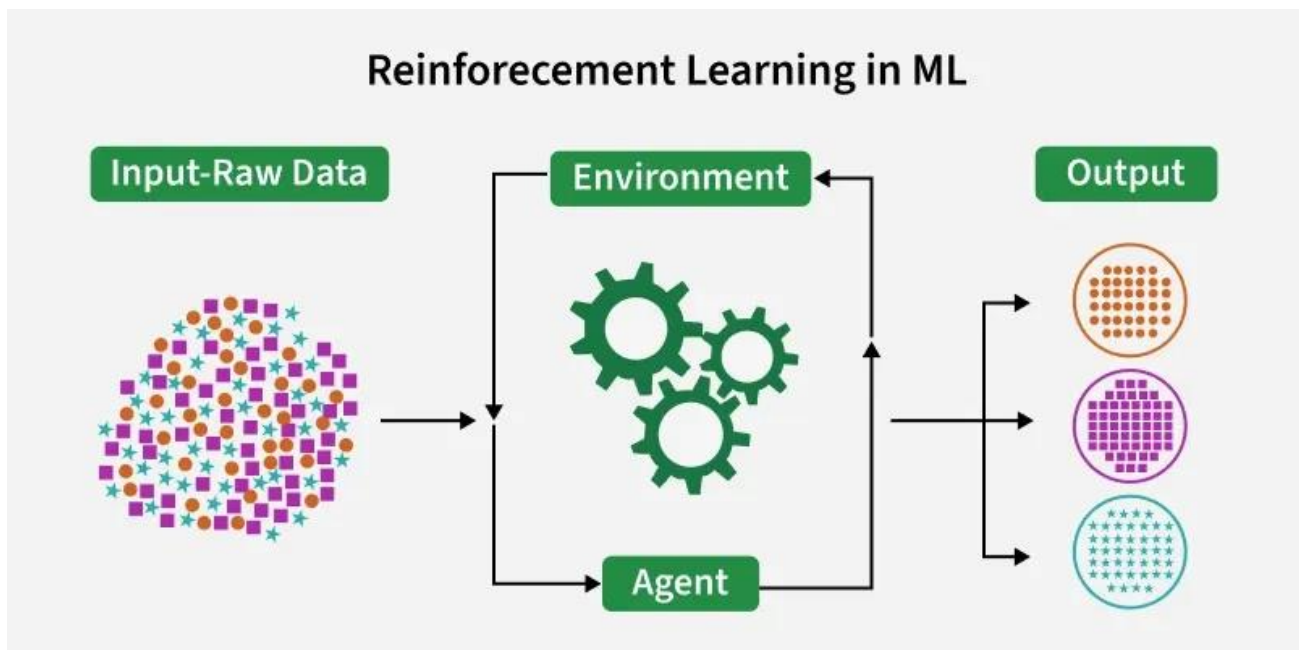


Mustahkamlash orqali o'qitish (Reinforcement Learning)



Mustahkamlash orqali o'qitish (RL) — bu mashinali o'qitishning bir tarmog'i bo'lib, u **agentlarning** umumiy mukofotlarni (*cumulative rewards*) maksimallashtirish maqsadida, **sinov va xato** (*trial and error*) usuli orqali qaror qabul qilishni o'rganishiga qaratilgan.

RL mashinalarga **muhit** (*environment*) bilan o'zaro aloqada bo'lish va o'z harakatlariga yarasha qayta aloqa (*feedback*) olish orqali o'rganish imkonini beradi. Ushbu qayta aloqa **mukofotlar** yoki **jarimalar** ko'rinishida keladi.

Oddiy tilda tushuntirish:

Buni xuddi yosh bolaning velosiped haydashni o'rganishi yoki itni o'rgatishga o'xshatish mumkin:

1. **Agent:** Velosiped haydayotgan bola.
2. **Muhit:** Yo'l va velosiped.
3. **Harakat:** Rulni burish yoki pedal aylantirish.
4. **Mukofot (Reward):** Muvozanatni saqlab, oldinga yurish (ijobiy).
5. **Jarima (Penalty):** Yiqilib tushish (salbiy).

Bola yiqilaverib (sinov va xato), oxiri yiqilmaslik uchun nima qilish kerakligini (mukofot olishni) o'rganib oladi.

RL g'oyasi shunga asoslanadiki, **agent** (o'rganuvchi yoki qaror qabul qiluvchi) biror maqsadga erishish uchun **muhit** bilan o'zaro aloqada bo'ladi. Agent harakatlarni amalga oshiradi va vaqt o'tishi bilan qaror qabul qilish jarayonini optimallashtirish (yaxshilash) uchun qayta aloqa (*feedback*) oladi.

- **Agent (Agent/Ijrochi):** Harakatlarni bajaruvchi va qaror qabul qiluvchi tomon.
- **Environment (Muhit):** Agent faoliyat yuritadigan dunyo yoki tizim.
- **State (Holat):** Agentning ayni paytdagi vaziyati yoki sharoiti.
- **Action (Harakat):** Agent amalga oshirishi mumkin bo'lgan yurishlar yoki qarorlar.
- **Reward (Mukofot):** Agentning harakatiga javoban muhitdan olinadigan natija yoki baho.

Asosiy tarkibiy qismlar (Core Components)

Keling, Mustahkamlash orqali o'qitishning 4 ta asosiy tarkibiy qismini ko'rib chiqamiz:

1. Policy (Siyosat yoki Strategiya)

- Agentning xulq-atvorini belgilaydi, ya'ni qaysi **holatda** qanday **harakat** qilish kerakligini xaritada ko'rsatgandek belgilab beradi.
- Bu oddiy qoidalar yoki murakkab hisob-kitoblar bo'lishi mumkin.
- **Misol:** Avtonom (haydovchisiz) mashina piyodani aniqlaganda (holat), to'xtashni (harakat) amalga oshirish qoidasi.

2. Reward Signal (Mukofot signali)

- RL muammosining asosiy **maqsadini** ifodalaydi.
- Agentga ijobiy (mukofot) yoki salbiy (jarima) qayta aloqa berish orqali uni to'g'ri yo'naltiradi.
- **Misol:** Haydovchisiz mashinalar uchun mukofotlar — to'qnashuvlarning yo'qligi, manzilga tezroq yetib borish va yo'l chizig'iga rioya qilish bo'lishi mumkin.

3. Value Function (Qiymat funksiyasi)

- Faqatgina darhol olinadigan mukofotni emas, balki **uzoq muddatli foydani** baholaydi.
- Kelajakdagi natijalarni hisobga olgan holda, hozirgi turgan holatning qanchalik "yaxshi" yoki "kerakli" ekanligini o'lchaydi.
- **Misol:** Mashina shunchaki tezroq yurish (qisqa muddatli yutuq) uchun xavfli manevr qilishdan qochadi, chunki uzoq muddatda xavfsizlik va samaradorlik muhimroq.

4. Model (Model)

- Harakatlarning natijasini oldindan bashorat qilish uchun muhitni **simulyatsiya** qiladi.

- Bu rejalashtirish va oldindan ko'ra bilish imkonini beradi.

- **Misol:** Xavfsizroq yo'nalishni tuzish uchun yo'ldagi boshqa mashinalarning ehtimoliy harakatini oldindan bashorat qilish.

Tushunish oson bo'lishi uchun qiyoslash:

Agar shaxmat o'yinini olsak:

- **Policy:** "Otni bu katakka yur" degan aniq qoida.

- **Value Function:** "Agar otini bu yerga yursam, 5 yurishdan keyin yutish ehtimolim 80% ga oshadi" deb vaziyatni baholash.

Mustahkamlash asosida o'qitishning (Reinforcement Learning) ishlash jarayoni

Agent o'z muhiti bilan qayta aloqa (feedback) halqasi orqali takroriy ravishda o'zaro ta'sir qiladi:

- Agent muhitning joriy **holatini** (*state*) kuzatadi.

- U o'z **strategiyasiga** (*policy*) asoslanib biror **harakatni** (*action*) tanlaydi va amalga oshiradi.

- Muhit yangi holatga o'tish va **mukofot** (*reward*) (yoki jarima) berish orqali javob qaytaradi.

- Agent qabul qilingan mukofot va yangi holat asosida o'z bilimlarini (strategiya, qiymat funksiyasi) yangilaydi.

Agent vaqt o'tishi bilan **umumiy mukofotni** (*cumulative reward*) maksimallashtirish maqsadida, **o'rganish** (*exploration* — yangi harakatlarni sinab ko'rish) va **foydalanish** (*exploitation* — ma'lum bo'lgan yaxshi harakatlarni qo'llash) o'rtasidagi muvozanatni saqlagan holda ushbu siklni takrorlaydi.

Bu jarayon matematik jihatdan **Markov Qaror Jarayoni (MDP)** sifatida ifodalanadi, bunda kelajakdagi holatlar oldingi voqealar ketma-ketligiga emas, balki faqat joriy holat va harakatga bog'liq bo'ladi.

Mustahkamlash asosida o'qitishni amalga oshirish (kodda)

Keling, labirint misolida mustahkamlash asosida o'qitishning ishlashini ko'rib chiqamiz:

1-qadam: Kutubxonalarni yuklash hamda Labirint, Boshlanish va Maqsad (target) ni aniqlash

- Biz `numpy` va `matplotlib` kabi kerakli kutubxonalarni yuklab olamiz (import qilamiz).
- Labirint 2 o'lchamli NumPy massivi sifatida ifodalanadi.
- Nol qiymatlari xavfsiz yo'llarni bildiradi; birlar esa agent chetlanishi kerak bo'lgan to'siqlardir.
- "Boshlanish" va "Maqsad" agentning qayerdan boshlashi va qayerga yetib borishi kerakligini belgilaydi.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

maze = np.array([
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
    [1, 1, 1, 0, 1, 0, 1, 1, 0, 1],
    [1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
    [1, 0, 1, 1, 1, 1, 1, 0, 1, 1],
    [1, 0, 1, 0, 0, 0, 0, 0, 1, 1],
    [1, 0, 1, 0, 1, 1, 1, 0, 1, 1],
    [1, 0, 1, 0, 1, 0, 0, 0, 1, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 0, 1],
    [1, 1, 1, 0, 1, 1, 1, 1, 0, 0]
])

start = (0, 0)
goal = (9, 9)
```

2-qadam: RL parametrlarini belgilash va Q-Jadvalni (Q-Table) initsializatsiya qilish

Biz RL (Reinforcement Learning) parametrlarini belgilab olamiz:

- `num_episodes`: Agent labirintdan o'tishga **harakat qiladigan** (urinishlar) soni.
- `alpha`: **O'rganish tezligi** (Learning rate). U yangi olingan ma'lumotlar eski ma'lumotlarning o'rnini qanchalik darajada egallashini (yangi bilimlarning qanchalik muhimligini) nazorat qiladi.
- `gamma`: **Chegirma omili** (Discount factor). U kelajakdagi mukofotlarga nisbatan **hozirgi (tezkor)** mukofotlarga ko'proq ahamiyat (vazn) berishni ta'minlaydi.
- `epsilon`: **O'rganish** (yangi yo'llarni sinash) va **foydalanish** (bilgan yo'ldan yurish) ehtimolligi. Agent ko'proq muhitni o'rganishi uchun bu qiymat boshida yuqori qilib belgilanadi.

Mukofotlar tizimi quyidagicha o'rnatiladi: to'siqlarga urilish uchun **jarima**, maqsadga yetish uchun **mukofot** beriladi va **eng qisqa yo'lni** topishga undash uchun har bir bosilgan qadamga ozgina jarima qo'llanadi.

- `actions` (harakatlar) mumkin bo'lgan yurishlarni belgilaydi: **chapga, o'ngga, tepaga, pastga**.
- `Q` — bu nollar bilan initsializatsiya qilingan (to'ldirilgan) **Q-Jadval** (Q-Table). U har bir "holat-harakat" (state-action) juftligi uchun **kutilayotgan mukofotlarni** o'zida saqlaydi.

```
num_episodes = 5000
alpha = 0.1
gamma = 0.9
epsilon = 0.5

reward_fire = -10
reward_goal = 50
reward_step = -1

actions = [(0, -1), (0, 1), (-1, 0), (1, 0)]

Q = np.zeros(maze.shape + (len(actions),))
```

3-qadam: Labirint tekshiruvi va Harakat tanlash uchun yordamchi funksiyalar

Biz yordamchi funksiyalarni aniqlab olamiz:

- `is_valid` — agent faqat labirint ichida harakatlanishini hamda to'siqlarga urilmasligini (to'siqlardan chetlanishini) ta'minlaydi.
- `choose_action` — **o'rganish** (*exploration* — tasodifiy harakat) va **foydalanish** (*exploitation* — o'rganilgan eng yaxshi harakat) strategiyasini amalga oshiradi.

```
def is_valid(pos):
    r, c = pos
    if r < 0 or r >= maze.shape[0]:
        return False
    if c < 0 or c >= maze.shape[1]:
        return False
    if maze[r, c] == 1:
        return False
    return True

def choose_action(state):
    if np.random.random() < epsilon:
```

```

        return np.random.randint(len(actions))
    else:
        return np.argmax(Q[state])

```

4-qadam: Agentni Q-Learning algoritmi yordamida o‘qitish

Biz agentni o‘rganishi uchun ko‘plab epizodlar (urinishlar) davomida mashq qildirib (o‘qitib) boramiz. Har bir epizod davomida agent harakatlarni tanlaydi va o‘z **Q-Jadvalini** (Q-Table) quyidagi Q-Learning yangilash qoidasi asosida yangilab boradi:

$$Q(s,a)=Q(s,a)+\alpha[r+\gamma\max_{a'}Q(s',a')-Q(s,a)]$$

Bunda:

- S : joriy holat (agentning labirintdagi joylashuvi).
- A : S holatda amalga oshirilgan harakat (masalan: chapga, o‘ngga, tepaga, pastga yurish).
- r : A harakati bajarilgandan so‘ng olingan mukofot.
- S' : A harakati bajarilgandan keyingi navbatdagi holat.
- α : o‘rganish tezligi. U yangi olingan ma’lumot eski ma’lumotni qay darajada yangilashini (o‘rnini egallashini) nazorat qiladi.
- γ : kelajakdagi mukofotlar uchun chegirma omili (faktori).

Ushbu tenglama Q-qiyamatni **joriy mukofot** va keyingi holatdan olinishi mumkin bo‘lgan **eng yaxshi kelajak Q-qiyamati** asosida yangilaydi.

```

rewards_all_episodes = []

for episode in range(num_episodes):
    state = start
    total_rewards = 0
    done = False

    while not done:
        action_index = choose_action(state)
        action = actions[action_index]

        next_state = (state[0] + action[0], state[1] + action[1])

        if not is_valid(next_state):
            reward = reward_fire
            done = True
        elif next_state == goal:

```

```

        reward = reward_goal
        done = True
    else:
        reward = reward_step

    old_value = Q[state][action_index]
    next_max = np.max(Q[next_state]) if is_valid(next_state) else 0

    Q[state][action_index] = old_value + alpha * \
        (reward + gamma * next_max - old_value)

    state = next_state
    total_rewards += reward

global epsilon
epsilon = max(0.01, epsilon * 0.995)
rewards_all_episodes.append(total_rewards)

```

5-qadam: O‘qitishdan so‘ng optimal yo‘lni aniqlash

Bu funksiya eng yaxshi yo‘lni topish (ajratib olish) uchun har bir holatdagi **eng yuqori Q-qiyamatlar** bo‘ylab harakatlanadi.

- U **maqsadga** yetib borilganda yoki hech qanday **mumkin bo‘lgan keyingi yurishlar** qolmaganda to‘xtaydi.
- **visited** (tashrif buyurilganlar) to‘plami sikllarga (bir joyda aylanib qolishga) tushib qolishning oldini oladi.

```

def get_optimal_path(Q, start, goal, actions, maze, max_steps=200):
    path = [start]
    state = start
    visited = set()

    for _ in range(max_steps):
        if state == goal:
            break
        visited.add(state)

        best_action = None
        best_value = -float('inf')

        for idx, move in enumerate(actions):
            next_state = (state[0] + move[0], state[1] + move[1])

            if (0 <= next_state[0] < maze.shape[0] and
                0 <= next_state[1] < maze.shape[1] and
                maze[next_state] == 0 and
                next_state not in visited):

                if Q[state][idx] > best_value:

```

```

        best_value = Q[state][idx]
        best_action = idx

    if best_action is None:
        break

    move = actions[best_action]
    state = (state[0] + move[0], state[1] + move[1])
    path.append(state)

return path

optimal_path = get_optimal_path(Q, start, goal, actions, maze)

```

6-qadam: Labirint, Robot yo‘li, Boshlanish va Maqsadni vizualizatsiya qilish

- Labirint va yo‘l tinchlantiruvchi yashil ranglar palitrasidan foydalangan holda tasvirlanadi (ko‘rsatib beriladi).
- **Boshlanish** va **maqsad** nuqtalari vizual tarzda alohida ajratib ko‘rsatiladi.
- Agentning yechimini namoyish etish uchun **o‘rganilgan yo‘l** aniq chizib beriladi.

```

def plot_maze_with_path(path):
    cmap = ListedColormap(['#eef8ea', '#a8c79c'])

    plt.figure(figsize=(8, 8))
    plt.imshow(maze, cmap=cmap)

    plt.scatter(start[1], start[0], marker='o', color='#81c784',
                edgecolors='black',
                s=200, label='Start (Robot)', zorder=5)
    plt.scatter(goal[1], goal[0], marker='*', color='#388e3c',
                edgecolors='black',
                s=300, label='Goal (Diamond)', zorder=5)

    rows, cols = zip(*path)
    plt.plot(cols, rows, color='#60b37a', linewidth=4,
            label='Learned Path', zorder=4)

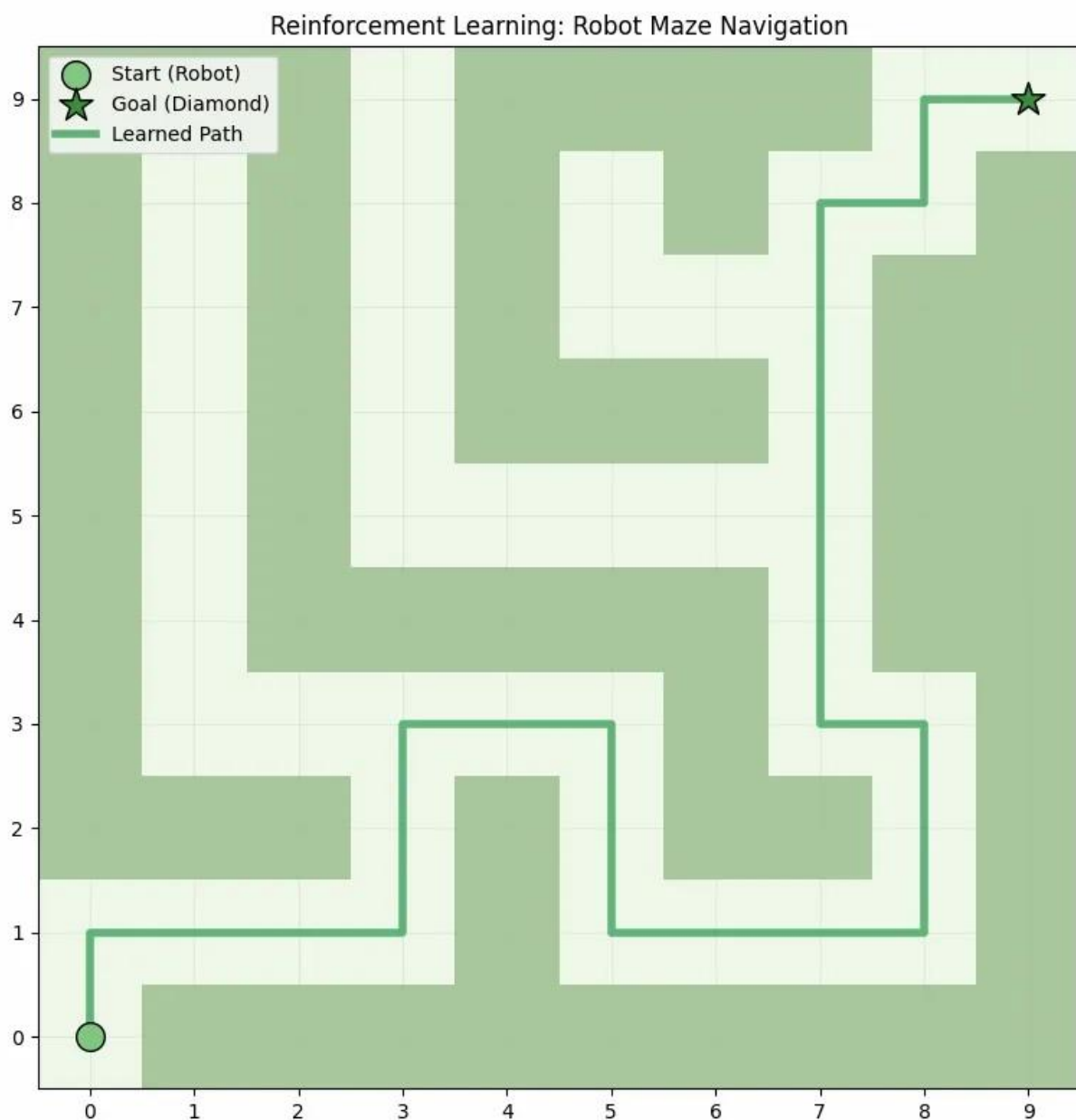
    plt.title('Reinforcement Learning: Robot Maze Navigation')
    plt.gca().invert_yaxis()
    plt.xticks(range(maze.shape[1]))
    plt.yticks(range(maze.shape[0]))
    plt.grid(True, alpha=0.2)
    plt.legend()
    plt.tight_layout()
    plt.show()

```



```
plot_maze_with_path(optimal_path)
```

Chiqish:



Ko‘rib turganimizdek, model to‘g‘ri yo‘lni topish orqali manzilga muvaffaqiyatli yetib bordi.

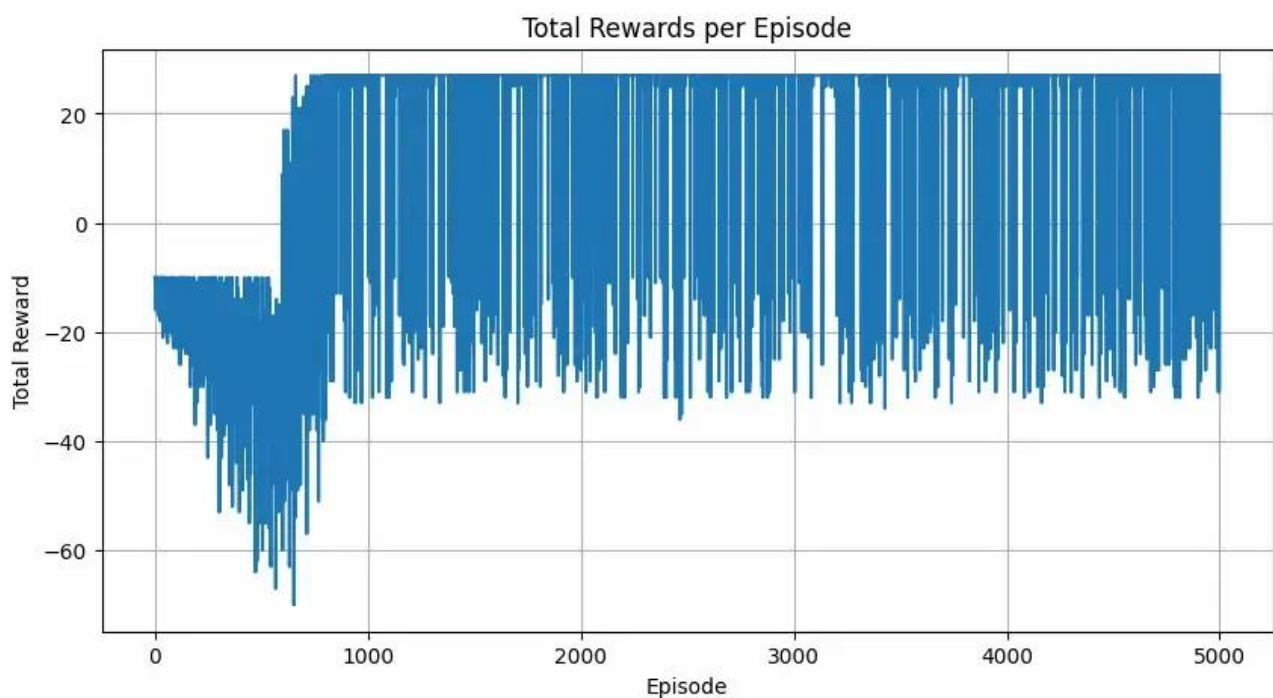
7-qadam: O‘qitish bo‘yicha mukofotlarni grafikda tasvirlash

- Ushbu grafik o‘qitish epizodlari (urinishlari) davomida agentning umumiy samaradorligi qanday yaxshilanib borishini ko‘rsatadi.
- Agent vaqt o‘tishi bilan o‘rganib borgani sari, biz umumiy mukofot tendensiyasining o‘sib borishini kuzatishimiz mumkin.

```
def plot_rewards(rewards):
    plt.figure(figsize=(10, 5))
    plt.plot(rewards)
    plt.title('Total Rewards per Episode')
    plt.xlabel('Episode')
    plt.ylabel('Total Reward')
    plt.grid(True)
    plt.show()
```

```
plot_rewards(rewards_all_episodes)
```

Chiqish:



Mustahkamlash turlari (Types of Reinforcements)

1. Ijobiy mustahkamlash (Positive Reinforcement):

Ijobiy mustahkamlash — bu ma'lum bir xulq-atvor tufayli yuzaga keladigan hodisa ushbu xulq-atvorning kuchi va takrorlanishini oshiradigan jarayondir. Boshqacha qilib aytganda, u xulq-atvorga ijobiy ta'sir ko'rsatadi.

- **Afzalliklari:** Samaradorlikni maksimallashtiradi va vaqt o'tishi bilan ijobiy o'zgarishlarni saqlab qolishga yordam beradi.
- **Kamchiliklari:** Haddan tashqari ko'p qo'llash "to'yinish" (excess states) holatiga olib kelishi va natijada samaradorlikni kamaytirishi mumkin.

2. Salbiy mustahkamlash (Negative Reinforcement):

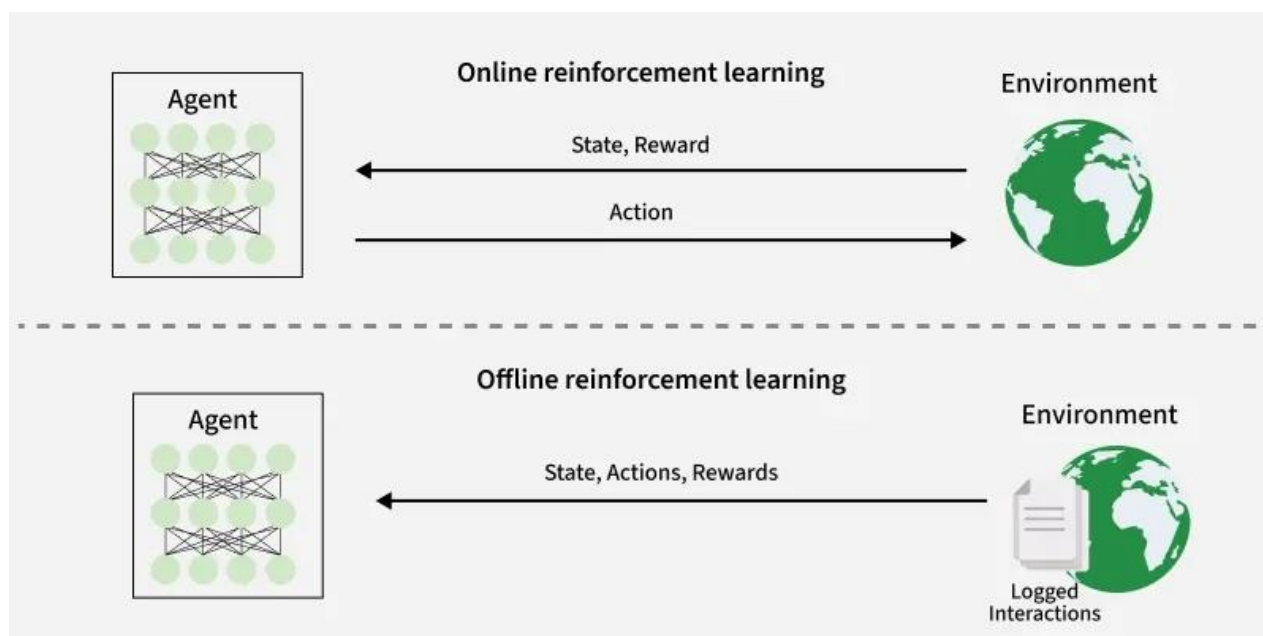
Salbiy mustahkamlash — bu salbiy holatning to'xtatilishi yoki uning oldi olinishi evaziga xulq-atvorning kuchayishi sifatida aniqlanadi.

- **Afzalliklari:** Xulq-atvor takrorlanishini oshiradi va minimal ishlash standartini ta'minlaydi.
- **Kamchiliklari:** U faqat jazolardan qochish uchun yetarli bo'lgan minimal harakatnigina rag'batlantirishi mumkin (maksimal natijaga intilmaslik).

Onlayn va Oflayn o'qitish (Online vs. Offline Learning)

Mustahkamlash asosida o'qitish (RL) agent o'z muhitidan ma'lumotlarni **qanday** va **qachon** olishiga qarab toifalarga ajratilishi mumkin. Bu usullar quyidagilarga bo'linadi:

- **Onlayn RL** (Online RL)
- **Oflayn RL** (Offline RL — shuningdek, **batch RL** deb ham ataladi).



Onlayn RLda agent muhit bilan **real vaqt rejimida faol o'zaro ta'sir qilish** orqali o'rganadi. U o'qitish jarayonida harakatlarni amalga oshirish va darhol qayta aloqani (feedback) kuzatish orqali yangi ("fresh") ma'lumotlarni to'playdi.

Oflayn RL agentni faqat **oldindan yig'ilgan statik ma'lumotlar to'plamida** o'qitadi. Bu ma'lumotlar boshqa agentlar, inson namoyishlari yoki tarixiy qaydlar (loglar) orqali hosil qilingan bo'ladi. Agent o'rganish jarayonida muhit bilan **umuman aloqa qilmaydi**.

Jihat (Aspect)	Onlayn RL	Oflayn RL
Ma'lumot olish	Muhit bilan to'g'ridan-to'g'ri, real vaqtda ishlash	Statik, oldindan yig'ilgan ma'lumotlar to'plami
Moslashuvchanlik	Yuqori, uzluksiz moslashib boradi	Cheklangan, ma'lumotlar to'plami qamroviga bog'liq

Mos kelishi	Muhitga kirish yoki uni simulyatsiya qilish mumkin bo'lganda	Muhit bilan ishlash qimmatga tushadigan yoki xavfli bo'lganda
Qiyinchiliklar	Ko'p resurs talab qiladi, potentsial xavfli bo'lishi mumkin	Taqsimot siljishi (distributional shift), kontrafaktual xulosa chiqarish muammolari

Qo'llanilishi (Application)

- **Robototexnika:** RL ishlab chiqarish kabi tuzilmalashgan muhitlarda vazifalarni avtomatlashtirishda ishlatiladi. Bunda robotlar harakatlarni optimallashtirish va samaradorlikni oshirishni o'rganadi.
- **O'yinlar:** Ilg'or RL algoritmlari shaxmat, Go va video o'yinlar kabi murakkab o'yinlar uchun strategiyalar ishlab chiqishda qo'llanilib, ko'p hollarda insonlarni mag'lub etmoqda.
- **Sanoat boshqaruvi:** RL sanoat operatsiyalarini, masalan, neft va gaz sanoatidagi qayta ishlash jarayonlarini real vaqtda sozlash va optimallashtirishga yordam beradi.
- **Shaxsiylashtirilgan o'qitish tizimlari:** RL o'quv kontentini individual o'rganish uslubiga moslashtirib, o'quvchining qiziqishi va samaradorligini oshiradi.

Afzalliklari (Advantages)

- Boshqa yondashuvlar o'z qoladigan **murakkab ketma-ket qaror qabul qilish muammolarini** hal qiladi.
- **Real vaqtda o'zaro ta'sir orqali o'rganadi**, bu esa o'zgaruvchan muhitga moslashish imkonini beradi.
- Nazorat ostida o'qitishdan (supervised learning) farqli o'laroq, **belgilangan (labeled) ma'lumotlarni talab qilmaydi**.
- Inson sezgisidan tashqari **yangi strategiyalarni kashf qila oladi** (innovatsiya).
- **Noaniqlik va stokastik (tasodifiy) muhitlar** bilan samarali ishlaydi.

Kamchiliklari (Disadvantages)

- **Hisoblash jihatidan og'ir**, katta hajmdagi ma'lumot va kuchli protsessor quvvatini talab qiladi.
- **Mukofot funksiyasini tuzish juda muhim**; noto'g'ri dizayn kutilmagan va noto'g'ri xatti-harakatlarga olib kelishi mumkin.
- An'anaviy usullar samaraliroq bo'lgan **oddiy masalalar uchun mos emas**.

- Xatolarni tuzatish (debug) va **natijalarni izohlash qiyin**, bu esa qarorlarning sababini tushuntirishni murakkablashtiradi.
- O‘rganishni optimallashtirish uchun **qidirish va foydalanish (exploration-exploitation)** o‘rtasidagi muvozanatni ehtiyotkorlik bilan saqlash talab etiladi.